From Digits to Characters: Two-Dimensional Rotary Coordinates for Length Generalization in a Compact Character-Level Transformer

Neithan Vargas Vargas
neithanvarvar@gmail.com

Abstract

We set out to train a small autoregressive language model that adds multi-digit integers correctly at lengths it has not seen. A standard 18.91M-parameter decoder with one-dimensional rotary positions (1D-RoPE), trained on 1–12-digit sums with a column scratchpad, solves short sums but scores 0/30 at every unseen length we tested (14–35 digits), with either a character or a 50k-token BPE vocabulary; three alternative architectures (looped, hierarchical, hybrid) do the same, and two others did not fit the task. Giving each token a two-dimensional coordinate, the decimal place p and the step τ within a column, through Decoupled 2D-RoPE changes the outcome: a 1.57M-parameter model trained on the same 1–12-digit range is exact on 150/150 sums of 14–25 digits and on 47/60 sums of 30–35 digits, and in controlled 100k-parameter runs the extrapolation limit moves with the coordinate range seen in training ($p_{\max} = N_{\text{train}} + k$). Our interpretation is that the coordinates turn place-value alignment into one fixed relation ($\Delta p = 0$) that does not change with length, so the model learns it once. Character-level models have no meaning-bearing subword embeddings and must assemble every word from characters, which prior work finds hard. We therefore asked whether the same relational layout, with p indexing the dialogue segment and τ the character within it, could support a compact character-level model that also handles words. The resulting Character-Based Transformer (CBT-18.95M) is exact on 210/210 random sums of 26–55 digits, all longer than any sum in training (254/270 up to 70 digits), and exact on all 74 prompts of three tool-use suites (26/26 phrasing, 18/18 out-of-distribution, and 30/30 copies of entities absent from the training corpus, against 4/30 for the pre-fine-tuning base). Its final fine-tuning data was broadened after earlier failures were seen, so the unseen-entity set is the cleaner evidence. To attribute these results we trained a 1D-RoPE twin with the same architecture, data and step count. It scores 0/270 on every unseen length from 26 to 70 digits, but matches the 2D model on tool use (73/74 exact calls against 74/74). The 2D coordinates are therefore responsible for length generalization, while the tool-use and copying results are not attributable to them.

1 Introduction

Small Transformers trained from scratch learn multi-digit addition within their training lengths but fail beyond them [1, 2, 3, 4, 5]. Addition is a convenient test of whether a model has learned an algorithm rather than a lookup over seen lengths: the procedure is a small finite-state machine applied once per column, and the answer is exactly checkable at any length. This paper reports a sequence of experiments that began with that question and ended with a compact character-level model. We present them in the order we ran them.

1. **A standard language model.** We trained an 18.91M-parameter decoder with 1D-RoPE [6] on 1–12-digit sums with an explicit column scratchpad [7, 8], using a character vocabulary and, separately, a 50k BPE vocabulary. It solved 4- and 8-digit sums, scored at most 14/30 at 12 digits, and scored 0/30 at every unseen length (Section 4).
2. **Other architectures.** Looped, hierarchical, hybrid recurrent, harmonic-register-machine and Abacus-looped variants [9, 10, 5], trained the same way, either scored 0/30 on every length from 12 digits up (three models) or did not fit the task (two models), and randomized 1D positions [11] did not help a 100k model (Section 5).
3. **Two-dimensional coordinates.** Assigning each token a coordinate (p, τ), place value and step within the column, and rotating queries and keys by both (Decoupled 2D-RoPE) made the models extrapolate. The extrapolation limit tracks the coordinate range seen in training, which suggests the model learns a fixed relation between positions of equal place value (Section 6).
4. **From digits to characters.** Character-level models must compose words from characters, which is a known weakness [12, 13, 14]. We asked whether the same relational layout helps a character model with words, and built the best small model we could around it: CBT-18.95M (Sections 7–9).
5. **A 1D control.** To find out which results the layout is responsible for, we trained a twin with standard 1D-RoPE on identical data. It does not extrapolate, but it matches the 2D model on tool use (Section 10).

Our contributions are:

- A re-evaluation of nine 1D-position checkpoints (standard, BPE and alternative architectures) on a common random-sum benchmark. None of them solves any unseen length.

- Evidence from controlled ablations that 2D place-value coordinates are the factor that enables extrapolation, and that the extrapolation limit is set by the trained coordinate range (Section 6).
- CBT-18.95M, an 18.9M-parameter character model that combines exact addition at lengths up to 55 digits (trained on at most 25) with tool calling and entity copying (Sections 8–9).
- A matched 1D-RoPE twin of CBT-18.95M, which shows that the 2D coordinates account for the arithmetic extrapolation (254/270 against 0/270 on unseen lengths) and not for the tool-use results (Section 10).

All results are single training runs; test sets and their sizes are given with every number.

2 Related Work

Length generalization in arithmetic. Transformers trained on short sums fail on longer ones [1, 2, 3], and how well they extrapolate depends strongly on the position encoding [4]. Scratchpads of intermediate steps help within the training range [7, 8].

Digit-aligned positions. Abacus embeddings [5] add to each digit's token embedding a learned embedding indexed by its position within its number, so that digits of equal place value share an index, and they report addition far beyond the training length. Index hints and positional formats [15, 16, 17] insert or reformat tokens to align digits, and randomized positions [11] train on position values drawn from a wider range. Our coordinates enter differently: they drive a rotary rotation inside attention, so only coordinate differences matter ($\Delta p = 0$ for aligned digits), a second coordinate carries the step within a column, and the same mechanism extends to dialogue segments. We did not compare against Abacus embeddings or index hints under our protocol, so we make no claim that our scheme is better.

Depth. Looped and universal Transformers [9, 10, 18] address length generalization through iterated depth. CBT-18.95M uses weight-tied recurrence, but our ablation found no addition benefit from it.

Character-level models. Character and byte models avoid tokenization artifacts but must compose every word from characters and tend to be harder to train than subword models [12, 13, 14, 19, 20, 21, 22].

3 Task and Evaluation Protocol

Addition as a state machine. For operands with digits a_i, b_i (column $i = 0$ is the units digit) the sum is produced by

$$s_i = (a_i + b_i + c_{i-1}) \bmod 10, \quad c_i = \left\lfloor \frac{a_i + b_i + c_{i-1}}{10} \right\rfloor, \quad (1)$$

with $c_{-1} = 0$. The same 200-entry transition table applies at every column, so a model that has learned it should work at any length.

Scratchpad format. Character models are trained on `A+B=<THINK>` followed by one block `(c+a+b)=c's` per column, from the units column up, then `</THINK>=S`. For example, `47+85=<THINK>(0+7+5)=12(1+4+8)=13</THINK>=132`. BPE models use the same steps in words (`Column 0: 7+5=12, digit 2, carry 1. ...`). The loss is masked on the prompt `A+B=`.

Benchmark. For each length d we draw 30 problems with two d-digit operands from a stratified generator: 70% uniform random digits, 15% dense carries (all digits 5–9), and 15% long carry cascades. Decoding is greedy. A problem counts as correct only if the answer after `</THINK>=` matches the true sum exactly. We report k/n with 95% Wilson intervals. Lengths inside the training range are labelled *seen*. Baseline traces up to 35 digits fit in their 512-token context, so truncation does not explain the failures below, with two exceptions noted where they occur: the place-value variant (Section 5) and the philosophy-mixed BPE model (Section 4).

4 Step 1: A Standard Language Model Does Not Extrapolate

Table 1 shows the 18.9M-parameter baselines ($L = 6$, $d_{\text{model}} = 512$, 8 heads, 1D-RoPE, 400M training tokens, 1–12-digit operands). All three solve short sums: the character model is 30/30 at 4 and 8 digits. None of them reaches 12 digits reliably, which is the longest length in training: the character model scores 0/30 there and the BPE word-chain model 14/30. Past 12 digits every baseline scores 0/30 at every length we tested, up to 35 digits. So the failure is not confined to unseen lengths; it begins at the edge of the training range, where the model has seen few examples, and is complete beyond it. For the philosophy-mixed BPE model, 42 of its 60 working traces at 30 and 35 digits exceeded the 512-token window, so those two zeros partly reflect truncation; its zeros at 14–25 digits do not.

How they fail. The wrong answers come from reading the wrong operand digit, not from arithmetic slips. For the 20-digit problem `64012498003866726193+44789680393999591374`, the character model's first column is `(0+9+4)=13`: it read the tens digit of the first operand (9) instead of its units digit (3). It then wrote only nine columns for a 20-column sum and answered `834569453`. At 12 digits (`320623293553+963365697522`) the first column is correct, the second column repeats it (`((0+3+2)=05(0+3+2)=05`, where the tens digits are 5 and 2), and the working stops one column early. With

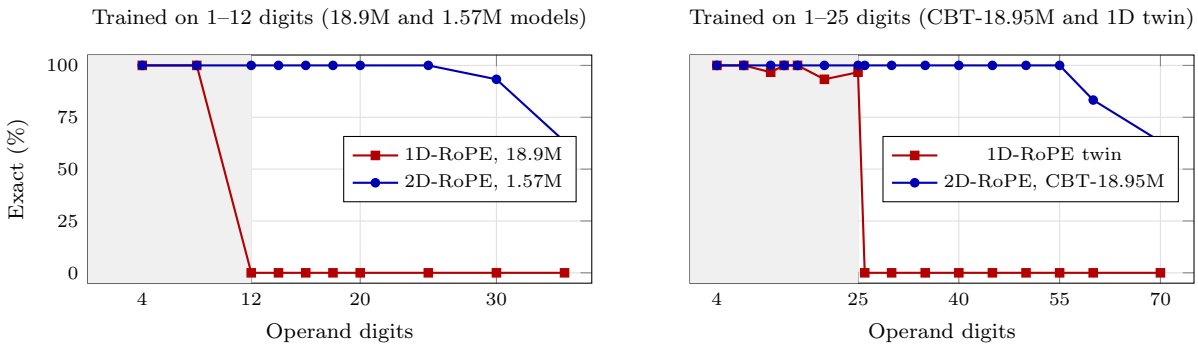


Figure 1: Standard 1D-RoPE models stop at the edge of their training range; 2D-RoPE models do not. Correct answers (%) out of 30 random sums per length. Shaded: lengths seen in training. Left: free-running decoding. Right: teacher-forced greedy agreement.

Table 1: A standard language model fails beyond its training length. Correct answers out of 30 random problems per digit length, greedy decoding, exact match. All models have 18.9M non-embedding parameters ($L = 6$, $d_{\text{model}} = 512$; the BPE models have 44.7M including the 50k-token embedding table) and were trained for 400M tokens on 1–12-digit sums with 1D-RoPE.

Model	Params	Seen (1–12 digits)			Unseen						
		4	8	12	14	16	18	20	25	30	35
Character, scratchpad	18.9M	30	30	0	0	0	0	0	0	0	0
BPE 50k, word chain-of-thought	18.9M	30	30	14	0	0	0	0	0	0	0
BPE 50k, 50% philosophy text	18.9M	21	18	0	0	0	0	0	0	0	0

a flat index t, the distance between a scratchpad column and the operand digits it must read grows with the number of digits. At unseen lengths the model has to attend at relative offsets it never saw in training, and it reads the wrong digit. Adding 50% philosophy text to the BPE model lowered its short-sum accuracy (21/30 at 4 digits, 18/30 at 8) and did not change the result beyond 12 digits.

5 Step 2: Other Architectures Do Not Fix It

We trained five other architectures on the same data and budget (Table 2): a looped Transformer that reuses two blocks six times [9, 10], a hierarchical fast/slow recurrent model, a hybrid of gated linear recurrence and attention, a harmonic neural register machine with a learned arithmetic unit (H-NRM), and an Abacus-looped model that adds place-value embeddings and shift augmentation [5] (ALSM). The looped, hierarchical and hybrid models behave like the plain Transformer: 30/30 at 4 and 8 digits, 0/30 at 12 and at every longer length. Recurrence and hierarchy do not make a flat index generalize.

The other two did not learn the task in the first place. H-NRM and ALSM score 0/30 even at 4 digits. A teacher-forced check on 24 four-digit sums, given the gold working, separates this from a decoding problem: the plain character model and the hybrid predict 100% of the next characters, H-NRM 92.6% (enough per-character error to break a working of 60 or more characters) and ALSM 47.7%. Our ALSM variant (four core blocks looped six times, Abacus-style place indices with random shift $k = 32$) is our own implementation, and we regard its result as a failed training run, not as evidence about the original Abacus method [5], which reports addition far beyond its training length. Two observations support this reading: the saved checkpoint scores higher when its place indices are zeroed (63–79% of post-prompt characters, against 31–59% with them), so it never learned to use them, and a scaled-down version of the same code ($d = 128$, two blocks, three loops, 1–6-digit sums) learns the task to 99.7–100% next-character accuracy within 600–1,200 steps (with shift $k = 0$ and $k = 32$; one $k = 8$ run reached loss 0.001 and then spiked). We could not recover the original training log, so the cause (we suspect optimization, since the recipe had no learning-rate warmup) is not established, and this baseline should be rerun before it is read as a negative result. A sixth baseline, a place-value variant trained with group relative policy optimization, scored 0/30 at every length. Its training traces were stored without line separators because the character vocabulary has no newline token in that format, so the terms ran together (`100050002006003070102`); we do not interpret this result.

Randomized 1D positions. We also trained four 100k-parameter character models ($L = 2$, $d_{\text{model}} = 64$)

Table 2: Alternative architectures do not extrapolate. Same protocol and training data as Table 1; correct answers out of 30. Parameters are non-embedding. The last row is the 2D model of Section 6, shown for comparison.

Model	Params	Seen (1–12 digits)			Unseen						
		4	8	12	14	16	18	20	25	30	35
Looped (2 blocks × 6)	6.3M	30	30	0	0	0	0	0	0	0	0
Hierarchical (fast 4, slow 3)	7.3M	30	30	0	0	0	0	0	0	0	0
Hybrid gated recurrence + attention	18.9M	30	30	0	0	0	0	0	0	0	0
Harmonic register machine (H-NRM)	18.9M	0	0	0	0	0	0	0	0	0	0
Abacus-looped (ALSM)	12.7M	0	0	0	0	0	0	0	0	0	0
Decoupled 2D-RoPE ($k = 32$)	1.57M	30	30	30	30	30	30	30	30	28	19

on the same 6,000 1–15-digit sums with a 1024-token context for 3,000 steps: standard 1D-RoPE, 1D-RoPE with a random offset $\Delta \sim \mathcal{U}\{0..512\}$, randomized sorted positions [11], and Decoupled 2D-RoPE with $k = 32$ (Section 6). Evaluated on 200 random problems per length, all three 1D variants scored 0/600 even on seen 8–15-digit sums (final losses 0.098, 0.096 and 0.232), while the 2D model reached 417/600 on seen lengths and 977/1,200 (81.4%) on unseen 16–40 digits (identical on re-evaluation). Retraining 1D-RoPE for 6,000 steps with a tuned learning rate reached 56/200 at 8 digits and 0/200 from 12 digits on (Table 5). In this setting the 1D models do not learn the task at all, so this comparison concerns trainability; the length-generalization failure of 1D models that do fit the task is the one shown in Table 1.

6 Step 3: Two-Dimensional Coordinates

Coordinates. Each token x_t receives a coordinate (p_t, τ_t) from a deterministic parser applied inside the forward pass, identically in training and generation (Table 3, Part A). An operand digit gets p equal to its place value ($p = 0$ for units) and $\tau = 0$. Inside the scratchpad, each `(` starts a new column, so p advances by one and τ counts the tokens since the `(` (capped at 9). After `</THINK>=`, the i-th answer digit gets $p = \max(0, C - 1 - i)$, where C is the number of scratchpad columns the model emitted. The model never predicts coordinates, but it controls column advancement by emitting `(`.

Decoupled 2D-RoPE. Within each head of dimension d_k, the first half of the query and key channels is rotated by p_t and the second half by τ_t:

$$\tilde{\mathbf{q}}_t = \begin{bmatrix} \mathcal{R}_\Theta(p_t)\, \mathbf{q}_{t,:d_k/2} \\ \mathcal{R}_\Theta(\tau_t)\, \mathbf{q}_{t,d_k/2:} \end{bmatrix}, \tag{2}$$

and likewise for $\tilde{\mathbf{k}}_t$, where $\mathcal{R}_\Theta(c)$ is the standard RoPE rotation with $\theta_j = 10000^{-4j/d_k}$.

Proposition 1. *If a scratchpad token and an operand digit share the same place value ($p_t = p_j$), the place half of their attention logit does not depend on the number of digits N; the step half depends only on $\tau_t, \tau_j \in \{0, \ldots, 9\}$, which are bounded for every N.*

This follows from RoPE depending only on coordinate differences: $\Delta p = 0$ gives the identity rotation. Under 1D-RoPE the same pair is separated by a distance that grows with N. The proposition is about one logit given correct coordinates; it does not show that the model computes carries correctly.

Coordinate shift masking. During training, every coordinate $p_t > 0$ is shifted by a per-sequence random offset,

$$p_t \leftarrow p_t + \Delta \cdot \mathbb{I}(p_t > 0), \quad \Delta \sim \mathcal{U}\{0, \ldots, k\}, \tag{3}$$

so that place coordinates up to $p_{\max} = N_{\text{train}} + k$ appear in training while the prompt start stays at $p = 0$. This is a coordinate-space version of randomized positions [11].

Result 1: a small 2D model extrapolates. We trained a 1.57M-parameter (non-embedding) model with Decoupled 2D-RoPE and shift masking $k = 32$ (embedding width 256, 4 heads, a two-level fast/slow recurrent core) on the same data as the baselines: 400M tokens of 1–12-digit sums with the same scratchpad. Table 2 (last row) shows it solves every sum we tested from 4 to 25 digits (240/240 over 8 lengths, including 12 digits, its longest training length), more than twice the training length, and scores 28/30 at 30 digits and 19/30 at 35. The 18.9M-parameter baselines with the same data and 12× more parameters score 0/30 at 14 digits. The accuracy begins to fall at 30 digits, below $p_{\max} = 12{+}32 = 44$, so the horizon is not sharp; we have no explanation for the gap beyond noting that the 100k models in Result 3 and CBT-18.95M in Section 9 also degrade somewhat before $p_{\max}$.

Sample efficiency. The same architecture trained for 250–600 steps on only N sums (Table 4) fits the training range with $N = 1000$ (29/30 at 4 digits, 30/30 at 8, 28/30 at 12) but extrapolates only briefly (22/30 at 14 digits, 14/30 at 16, 1/30 at 25). With $N = 100$ and $N = 10$ it did not fit the task (2/30 and 0/30 at 4 digits). So the extrapolation we see with full training is not free: it

Table 3: Coordinates assigned by the CBT parser. *Part A, arithmetic*: prompt `47+85=`, scratchpad `<THINK>(0+7+5)=12(1+4+8)=13</THINK>`, answer `=132`. Digits of the same place value share p in the prompt and scratchpad, whatever the operand length. In the answer, p counts down from the last scratchpad column, so when the sum has a carry-out digit (as here, `132`) the answer coordinates are one place lower than the true place values; the model is exact on such sums (Section 9). *Part B, dialogue*: p is the index of the most recent segment tag and τ counts characters since that tag.

Token	4	7	+	8	5	=	<THINK>	(	0	+	7	+	5	)	=	1	2	</THINK>	=	1	3	2
1D index t	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	28	29	30	31	32
Place p_t	1	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0
Step τ_t	0	0	0	0	0	0	0	0	1	2	3	4	5	6	7	8	9	0	0	0	0	0

Token	<USER>	W	h	a	t...	<THINK>	I		w...	<CALL>	s	e	a	r	c...	<RESULT>	{	"...	<ANSWER>	I	t	...
Segment p_t	0	0	0	0	0	1	1	1	1	2	2	2	2	2	2	3	3	3	4	4	4	4
Character τ_t	0	1	2	3	4	0	1	2	3	0	1	2	3	4	5	0	1	2	0	1	2	3

Table 4: Sample efficiency of the 2D model. Correct answers out of 30 per length; N training sums with up to 11 digits (the subsets contain no 5–7-digit operands), 250–600 steps.

N	4	8	12	14	16	20	25	30
10	0	0	0	0	0	0	0	0
100	2	0	0	0	0	0	0	0
1000	29	30	28	22	14	4	1	0

needs the model to fit the training range first, and 1000 examples are enough only for a short margin.

Result 2: 2D coordinates are the factor that matters. In a 100k-parameter factor ablation (character vocabulary, $L = 2$, $d_{\text{model}} = 64$, 2,000 steps, 1–12-digit training, evaluated on a fixed 14-case suite), replacing 1D-RoPE with 2D-RoPE lowered the final training loss from 0.186 to 0.0011 and raised the score from 3/14 to 11/14, and from 0/8 to 5/8 on the unseen 14–20-digit cases. Adding weight-tied recurrence ($R = 3$) changed nothing (11/14 with or without, and 2/14 for 1D with recurrence), and adding shift masking ($k = 16$) also gave 11/14. With 14 cases, differences between the 2D variants are not meaningful; we include the ablation for the 1D versus 2D contrast and because it is the only test of recurrence we ran.

Result 3: the extrapolation limit is set by the trained coordinate range. Table 5 varies the shift range k for 100k 2D models trained on 1–15 digits. The point where accuracy collapses moves with $p_{\max} = 15+k$: past 16–25 digits for $k = 0$, 35–40 for $k = 16$, 45–50 for $k = 32$ and 55–60 for $k = 64$. Setting $\tau = 0$ (p only) keeps 97.5% at 30 digits but falls faster near the limit (41.5% vs. 58.0% at 40 digits), so the place coordinate carries most of the effect and the step coordinate adds robustness.

Interpretation. With a flat index, "read the operand digit of the current column" is a different relative offset for every column and every problem length, so a model trained on short problems learns offsets that do not exist at longer lengths. With place coordinates the same instruction becomes one relation, attend where $\Delta p = 0$, identical for every column and length. We think the 2D models extrapolate because they learn this single fixed relation, in effect memorizing a linear relation in coordinate space rather than a set of length-specific distances. The relation only holds for coordinate values the model has seen, which is why the limit follows $p_{\max}$ rather than the training length. This account is consistent with the shift sweep and the place-only ablation; we have not tested it directly, for example by probing attention patterns at unseen lengths.

7 Step 4: From Digits to Characters

Subword vocabularies give a language model ready-made units that carry meaning. A character-level model has no such units: every word must be assembled from characters, sequences are several times longer, and prior work finds character models harder to train and weaker or slower than subword models at equal scale [12, 13, 14]. Character and byte models remain attractive because they have no tokenization artifacts and a tiny vocabulary [19, 20, 21, 22]. For arithmetic in particular, BPE merges digits of different place values into one token, which breaks the column alignment above.

The addition results suggested a question for character models. If a fixed relation in coordinate space lets a model align digits regardless of length, can the same kind of coordinate help a character model treat a word as a unit? We extended the parser to dialogue (Table 3, Part B): p indexes the segment (`<USER>`, `<THINK>`, `<CALL>`, `<RESULT>`, `<ANSWER>`) and τ counts characters within it. Copying an entity from the user message into a tool call then becomes a relation between two segments, from $p = 0$ to $p = 2$, at matching character offsets within the entity, analogous to $\Delta p = 0$ in addition. The hypothesis we test is that a compact character model built on this layout can do both exact arithmetic and usable word-level tasks such as tool routing and entity copying. Section 10 tests whether the layout is responsible for the language results by training a 1D twin on the same data.

Table 5: The extrapolation limit tracks the trained coordinate range. 100k-parameter character models trained on 1–15-digit sums with a 1024-token context, 200 random problems per length. Columns are 2D-RoPE with shift range k, a place-only model ($\tau = 0$, $k = 32$), and 1D-RoPE retrained for 6,000 steps. Values reproduced exactly on re-evaluation.

Digits	2D, $k = 0$ $p_{\max} = 15$	2D, $k = 16$ $p_{\max} = 31$	2D, $k = 32$ $p_{\max} = 47$	2D, $k = 64$ $p_{\max} = 79$	p only, $k = 32$ $p_{\max} = 47$	1D retrained
8 (seen)	200/200	200/200	120/200	199/200	199/200	56/200
12 (seen)	200/200	199/200	146/200	198/200	200/200	0/200
15 (seen)	200/200	200/200	170/200	199/200	200/200	0/200
16	155/200	200/200	172/200	200/200	200/200	0/200
20	34/200	200/200	170/200	196/200	199/200	0/200
25	0/200	199/200	178/200	187/200	198/200	0/200
30	0/200	197/200	171/200	177/200	195/200	0/200
35	0/200	121/200	151/200	166/200	170/200	0/200
40	0/200	0/200	116/200	138/200	83/200	0/200
45	0/200	0/200	61/200	120/200	19/200	0/200
50	0/200	0/200	0/200	82/200	0/200	0/200
55	0/200	0/200	0/200	40/200	0/200	0/200
60	0/200	0/200	0/200	2/200	0/200	0/200

8 CBT-18.95M

Architecture. CBT-18.95M is a decoder with $L = 6$ blocks, $d_{\text{model}} = 512$, 8 heads ($d_k = 64$), pre-LayerNorm, no biases, tied input and output embeddings, and a 1024-token context. The six blocks are applied $R = 3$ times with shared weights (18 effective layers) using a residual update scaled by $1/\sqrt{R}$. We kept recurrence as a parameter-efficient way to add depth, not because addition needs it: it gave no benefit in the 100k ablation, and we did not test $R = 1$ at this scale. The model has 18,944,512 parameters, 18,887,680 of them outside the embedding table, or about 36 MB in FP16.

Vocabulary. 111 tokens: 97 characters (the 95 printable ASCII characters, tab and newline) and 14 control tokens (`<PAD>`, `<BOS>`, `<EOS>`, `<UNK>` and opening and closing tags for `USER`, `THINK`, `CALL`, `RESULT`, `ANSWER`). Tags are matched greedily; everything else is one token per character.

Training. All stages ran on NVIDIA T4 GPUs with FP16 mixed precision, with loss masked on prompts and on `<RESULT>` payloads.

1. *Pretraining* (45,059 steps, 247.3M tokens, stopped by a time limit, $k = 32$): an equal mix of 1–15-digit addition (random sums of equal and unequal operand lengths, carry cascades such as $99\ldots9+b$, and powers of ten) and structured tool dialogues in six categories.
2. *Search-copy fine-tuning* (7,000 steps): dialogues that require copying a search query from the user message, 65% of them procedurally generated pseudowords so the entity cannot be memorized, with arithmetic and tool replay.
3. *All-tools fine-tuning* (4,000 steps, final loss 0.0075): the same copying task across nine tool schemas with 10% arithmetic replay. This checkpoint is the *pre-SFT base*.
4. *Intermediate fine-tuning* (12,000 steps, final loss 0.0034, $k = 45$): 45% multi-tool dialogues, 35% search copy and 20% arithmetic replay with 1–25-digit operands. This is the *intermediate model*.
5. *Final fine-tuning* (23,927 steps, stopped by a 6-hour budget, $k = 45$): a decontaminated online stream of 35% search copy, 20% arithmetic replay (1–25 digits) and the rest multi-tool and chat dialogues, with loss weight 2 on tool-call arguments. Its phrasings were broadened after the intermediate model's failures were seen. Held-out tool exactness at the end of training was 63/64 and held-out math 100%. **This is CBT-18.95M.**

Because of the replay, sums of up to 25 digits are seen in training and $p_{\max} = 25 + 45 = 70$. All test prompts and entities were checked against the fine-tuning corpus. Section 9 evaluates CBT-18.95M and, where noted, the earlier stages; Section 10 compares it with a 1D-RoPE twin.

9 Results of CBT-18.95M

All results in this section were obtained on CPU from the checkpoints, with greedy decoding. Tool calls are free-running and scored by exact match of the emitted call. Arithmetic is scored by teacher-forced greedy agreement: a sum counts as correct only if the greedy argmax equals the gold scratchpad and answer at every position after the prompt, which is the condition for free-running greedy decoding to reproduce the gold sequence. Each length has 30 random sums (70% uniform digits, 15% dense carries, 15% cascades such as $99\ldots9+b$).

Arithmetic. Table 6 and Figure 1 (right) show that CBT-18.95M is exact on all 150 sums at 4–16 digits, on

Table 6: Addition of CBT-18.95M (2D-RoPE) and its 1D-RoPE twin. Correct sums out of 30 per length. Training covers 1–25 digits; the twin is described in Section 10.

Digits	Status	2D-RoPE	1D-RoPE
4, 8, 12, 14, 16	seen	150/150	149/150
20, 25	seen	60/60	57/60
26	unseen (+1)	30/30	0/30
30, 35, 40, 45, 50, 55	unseen	180/180	0/180
60	unseen	25/30	0/30
70	unseen	19/30	0/30
26–70 pooled	unseen	254/270	0/270

all 60 at 20 and 25 digits, and on all 210 at 26–55 digits (Wilson 95% CI 98.2–100%), which are longer than any sum in training. Accuracy falls at 60 digits (25/30) and 70 digits (19/30), still inside $p_{\max} = 70$ (254/270 pooled over 26–70 digits, CI 90.6–96.3%). Localizing the first wrong token of the 16 failures at 55–70 digits shows that all are uniform-digit sums with an error inside the scratchpad, at columns 59–70 (token positions 705–840). There is no error in columns 1–58 on any problem, and every cascade (17/17) and dense-carry sum (9/9) at 55–70 digits is exact, so the failures are local errors in a column's carry or digit, not the digit drift of the 1D baselines. At 80 and 100 digits the full trace would be about 1,150 and 1,430 tokens, longer than the 1,024-token context. For reference, the intermediate model scored free-running on a different random sample was exact on 240/240 sums of 16–50 digits, 14/20 at 60 and 6/15 at 70 digits.

Figure 2 varies the two operand lengths independently, with 10 random sums per cell. The 2D model is exact on all 250 cells inside the trained range (both operands up to 25 digits) and on 644 of the 750 sums that have at least one longer operand; the twin gets 234/250 inside the range and 0/750 outside it. The 2D model is perfect for operands up to 50 digits in both positions, and its errors are concentrated where an operand has 60 or 70 digits (195/250 when both exceed 25 digits, with the weakest row and column at 70). So unequal operand lengths cause no extra difficulty beyond the length of the longer operand.

Tool use. The phrasing suite has 26 prompts with paraphrases and invented entities over weather, time, unit conversion, search, calculator, conversational reply and four nested-schema tools (database query, HTTP fetch, calendar event, file manager), whose schemas occur in the training data. The out-of-distribution (OOD) suite has 18 prompts whose entities do not occur in the fine-tuning corpus (audited: 0 overlaps). The unseen-copy suite has 30 prompts in five groups (historical figures, cities, science terms, concepts, invented words), none of whose entities appear in any training stage. Table 7 reports routing (right tool) and exact match (right tool and argument string).

Table 7: Tool calls across fine-tuning stages. Right tool / exact call. The base is the pre-SFT checkpoint, Intermediate is the model after the first fine-tuning round, 2D is CBT-18.95M and 1D its twin. The base was scored only on unseen copy. The final fine-tuning data was broadened against the phrasing and OOD suites (Section 8).

Suite	Base	Interm.	2D	1D
Phrasing (26)	–	21 / 13	26 / 26	25 / 25
OOD entities (18)	–	18 / 10	18 / 18	18 / 18
Unseen copy (30)	28 / 4	29 / 17	30 / 30	30 / 30

CBT-18.95M routes and copies correctly on all 74 prompts (Wilson 95% lower bounds 87%, 82% and 89% for the three suites). The unseen-copy set is the cleanest evidence, since none of its entities occur in any training corpus: all five groups are 6/6, and the exact-copy rate is above 50% (one-sided binomial $p = 9.3 \times 10^{-10}$). The phrasing and OOD suites are less clean because the final fine-tuning data was broadened after the intermediate model's failures on them were seen.

Effect of the fine-tuning stages on copying. Copying is the weak part of a character model, and the earlier stages show it. On the unseen-copy suite the pre-SFT base copies 4/30 entities exactly and the intermediate model 17/30 (routing roughly constant, 28 and 29 of 30). The base's errors are mostly fragments (`Belo` for Belo Horizonte, `Cai` for Chiang Mai, `quarks` for quarks and leptons); the intermediate model's are mostly one- or two-character slips (`Barch Spinoza`, `Ada Nonath`, `Chiang Maix`) and, on the OOD and phrasing suites, corrupted spellings (`Auccra` for Accra, `Rerevan` for Yerevan, `zggdrasil`) and truncated names (`Santo` for Santo Domingo). The intermediate model also routed all four nested-schema phrasings wrongly (three of them to `search`). CBT-18.95M gets every one of these cases exact, including the city, time-zone and multi-word cases that the intermediate model corrupted. This is the word-formation weakness of character models discussed in Section 7, and the result shows that more targeted training data removes it at this scale.

10 Control: A 1D-RoPE Twin

The experiments so far leave open which of CBT-18.95M's results the coordinate layout is responsible for. We trained a twin that differs in position encoding only: standard RoPE over the whole head dimension at positions $0, \ldots, T-1$, with the same architecture (18,944,512 parameters), vocabulary, intermediate recipe (12,000 steps) and final recipe (23,927 steps, step-matched). The final-stage corpus is byte-identical after decompression (SHA256 of the text matches). A uniform shift of positions is an exact no-op under RoPE, so the twin has no shift augmentation, and the compar-

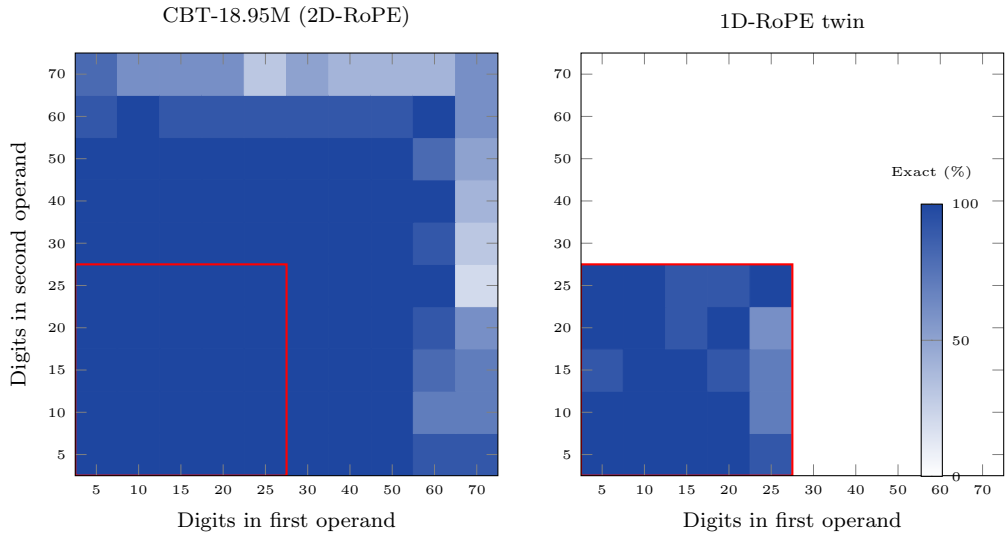


Figure 2: Addition accuracy by operand lengths. Each cell is 10 random sums whose first and second operands have the stated numbers of digits (teacher-forced greedy agreement). Red box: operand lengths seen in training (at most 25 digits). The 2D model stays accurate far outside the box; the 1D twin is accurate only inside it.

ison is between the full 2D scheme and plain 1D-RoPE. The lineages also differ before the intermediate stage: the 2D model's pretraining stopped at step 45,059 while the twin completed 50,000 steps, and the 2D model also received the search-copy and all-tools stages (11,000 steps, neither with arithmetic) that the twin skipped. Both models were scored with the harness of Section 9.

Arithmetic. The twin drops from 29/30 at 25 digits to 0/30 at 26, one digit past the training maximum, and scores 0 on every unseen length (Table 6, 0/270 pooled, CI 0.0–1.4%). Localizing the first wrong token of the twin's 150 failures at 26, 30, 40, 55 and 70 digits shows that all of them lie inside the scratchpad. For the 124 uniform and dense sums the first error is in column 1 (93), 2 (29) or 3 (2): the twin misreads an operand digit at the start of the scratchpad. For the 26 carry cascades it is in columns 22–27 and always writes `0` for an operand digit `9`, as if the operand of nines ended near the 25-digit training maximum. This contrasts with the 2D model, whose first error is never before column 59. The pattern is consistent with the Proposition in Section 6: under 1D positions the distance between a scratchpad column and the operand digit it must read grows with the operand length and leaves the trained range, while under 2D coordinates it does not. We did not test that explanation directly, for example with attention maps, and we did not find a single consistent wrong-place offset in the misread digits.

Tool use. Table 7 shows no such gap. The twin routes and copies as well as the 2D model on all three suites (73/74 against 74/74 exact calls); its one failure, the HTTP-fetch phrasing, picks the right tool and endpoint and then breaks the `params` dictionary so that the call never closes.

This changes how Section 7 should be read. The final fine-tuning data, not the layout, produced the gain in tool use and copying: the intermediate model rose from 17/30 to 30/30 exact copies on the unseen set (entities absent from its corpus), and a 1D model trained on the same data reached the same score. The hypothesis that segment and character-offset coordinates help a character model assemble words is therefore not supported at this scale and sample size; a character model with plain 1D-RoPE copies unseen entities just as well once the data cover the failure modes. What the layout does contribute is arithmetic length generalization. The other direction also holds: a character-level model with 1D positions can be a competent tool user and still be unable to add a 26-digit sum.

11 Discussion and Limitations

Limitations.

- The coordinates are assigned by a rule-based parser, not learned, and the parser assumes the `A+B=` format. The arithmetic result is conditional on correct coordinates and covers only addition with a fixed scratchpad.
- The 1D control (Section 10) compares the full 2D scheme, coordinates and shift augmentation together, with plain 1D-RoPE, so it does not show which component is necessary. The two lineages also differ before the intermediate stage (the twin had 50,000 pretraining steps against 45,059 and skipped two tool fine-tuning stages with no arithmetic), each model is a single run, and the mechanism is inferred from where the twin fails, not tested directly.

- The final fine-tuning data were broadened after the intermediate model's failures on the phrasing and OOD suites were seen, so CBT-18.95M's scores on those suites partly reflect targeted coverage. The unseen-copy set, whose entities are absent from the corpus, is the cleaner evidence. The twin shares this data, so the 2D/1D comparison is unaffected. The ceiling on the three suites (100% observed, lower Wilson bounds of 87%, 82% and 89%) also limits how small a difference they can detect.
- Two of the alternative architectures (H-NRM and ALSM) did not fit the task, so Table 2 cannot show that those designs fail to extrapolate, only that they did not train here.
- Every configuration is a single run, and several test sets are small (14 cases for the factor ablation, 26 and 18 for tools, 30 per length for arithmetic).
- Extrapolation is bounded by $p_{\max}$, and CBT-18.95M already degrades below it (60 and 70 digits).
- Generation is character by character with three recurrent passes, so per-token compute is about that of a 57M-parameter dense model; the 36 MB figure covers weights only.

12 Conclusion

We asked a small autoregressive model to add integers longer than any it had seen. Standard 1D-RoPE Transformers with a character or BPE vocabulary solved short sums and then scored 0/30 at every length past 12 digits, and three recurrent or hierarchical variants did the same. Giving each token a two-dimensional coordinate for place value and step let a 1.57M-parameter model add 14–25-digit sums without error (150/150) after training on 1–12 digits, and controlled 100k-parameter runs show that its limit follows the range of coordinates seen in training. We read this as the model learning one fixed relation between digits of equal place value, which is a hypothesis we have not tested directly.

Carrying the same layout over to dialogue segments gave CBT-18.95M, a character-level model with a 111-token vocabulary. It adds sums of 26–55 digits exactly (210/210, with 254/270 up to 70 digits) after training on at most 25 digits, and its tool calls are exact on all 74 prompts of three suites, including 30/30 copies of entities absent from every training corpus. Its earlier stages showed the weakness expected of a character model: they picked the right tool but misspelled words they copied (4/30 and 17/30 exact copies), which targeted fine-tuning data removed.

The control settles what the layout is responsible for. A 1D-RoPE twin of CBT-18.95M, trained on identical data for the same number of steps, scored 0/270 on sums of 26–70 digits, against 254/270 for the 2D model, and it failed from the first column of the scratchpad. On tool use and entity copying it scored 73/74 against 74/74. So the hand-assigned two-dimensional coordinates gave a compact character model exact length generalization on addition, which every 1D model we trained lacked, while the language behaviour came from the training data and not from the layout. We did not find evidence that segment and character-offset coordinates help a character model form words.

Code, Models and AI Use

Code, data generators and evaluation scripts are at github.com/NeithanT/CBT; CBT-18.95M is released at huggingface.co/Neithanvarvas/CBT as `champion_final.pt` and stored locally as `checkpoints/cbt/cbt_18m_2d_rope_R3_k45_sft.pt` (SHA256 `c6fab36e...b9e5`). The 1D-RoPE twin is `cbt_18m_1d_rope_R3_k45_sft.pt` (SHA256 `4fa72c78...ef1e`); both are `torch.save` files. Per-problem results are in the `results/` directory of the repository. Parts of the engineering, evaluation code and manuscript preparation were done with AI coding assistants (Google Gemini in the Antigravity environment, and Claude Code); the author verified the experiments and conclusions.

References

[1] R. Nogueira, Z. Jiang, and J. Lin. Investigating the limitations of transformers with simple arithmetic tasks. arXiv preprint arXiv:2102.13019, 2021.

[2] S. Jelassi, S. d'Ascoli, C. Domingo-Enrich, Y. Wu, Y. Li, and F. Charton. Length generalization in arithmetic transformers. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 36, 2023.

[3] N. Dziri, X. Lu, M. Sclar, X. L. Li, L. Jiang, B. Y. Lin, S. Welleck, P. West, C. Bhagavatula, R. Le Bras, et al. Faith and fate: Limits of transformers on compositionality. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 36, 2023.

[4] A. Kazemnejad, I. Padhi, K. Natesan Ramamurthy, P. Das, and S. Reddy. The impact of positional encoding on length generalization in transformers. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 36, 2023.

[5] S. McLeish, A. Bansal, A. Stein, N. Jain, J. Kirchenbauer, B. R. Bartoldson, B. Kailkhura, A. Bhatele, J. Geiping, A. Schwarzschild, and T. Goldstein.

Transformers can do arithmetic with the right embeddings. In *International Conference on Machine Learning (ICML)*, 2024.

[6] J. Su, M. Ahmed, Y. Lu, S. Pan, W. Bo, and Y. Liu. Roformer: Enhanced transformer with rotary position embedding. *Neurocomputing*, 568:127063, 2024.

[7] M. Nye, A. J. Andreassen, G. Gur-Ari, H. Michalewski, J. Austin, D. Bieber, D. Dohan, A. Lewkowycz, M. Bosma, D. Luan, et al. Show your work: Scratchpads for intermediate computation with language models. arXiv preprint arXiv:2112.00114, 2021.

[8] J. Wei, X. Wang, D. Schuurmans, M. Bosma, F. Xia, E. Chi, Q. V. Le, D. Zhou, et al. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 35, pages 24824–24837, 2022.

[9] M. Dehghani, S. Gouws, O. Vinyals, J. Uszkoreit, and L. Kaiser. Universal transformers. In *International Conference on Learning Representations (ICLR)*, 2019.

[10] A. Giannou, S. Rajput, J.-y. Sohn, K. Lee, J. D. Lee, and D. Papailiopoulos. Looped transformers as programmable computers. In *International Conference on Machine Learning (ICML)*, pages 11398–11442, 2023.

[11] A. Ruoss, G. Deletang, T. Genewein, J. Grau-Moya, R. Csordas, M. Bennani, S. Legg, and J. Veness. Randomized positional encodings boost length generalization of transformers. In *Annual Meeting of the Association for Computational Linguistics (ACL)*, 2023.

[12] R. Al-Rfou, D. Choe, N. Constant, M. Guo, and L. Jones. Character-level language modeling with deeper self-attention. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2019.

[13] Jindřich Libovický, Helmut Schmid, and Alexander Fraser. Why don't people use character-level machine translation? In *Findings of the Association for Computational Linguistics: ACL 2022*, pages 2470–2485, 2022.

[14] Sabrina J. Mielke, Zaid Alyafeai, Elizabeth Salesky, Colin Raffel, Manan Dey, Matthias Gallé, Arun Raja, Chenglei Si, Wilson Y. Lee, Benoît Sagot, and Samson Tan. Between words and characters: A brief history of open-vocabulary modeling and tokenization in NLP. arXiv preprint arXiv:2112.10508, 2021.

[15] Y. Zhou, U. Alon, X. Chen, X. Wang, R. Agarwal, and D. Zhou. Transformers can achieve length generalization but not robustly. arXiv preprint arXiv:2402.09371, 2024.

[16] R. Shen, S. Bubeck, R. Eldan, Y. T. Lee, Y. Li, and Y. Zhang. Positional description matters for transformers arithmetic. arXiv preprint arXiv:2311.14737, 2023.

[17] S. Duan and Y. Shi. From interpolation to extrapolation: Complete length generalization for arithmetic transformers. arXiv preprint arXiv:2310.11984, 2023.

[18] Y. Fan, Y. Du, K. Ramchandran, and K. Lee. Looped transformers for length generalization. arXiv preprint arXiv:2409.15647, 2024.

[19] L. Xue, A. Barua, N. Constant, R. Al-Rfou, S. Narang, M. Kale, A. Roberts, and C. Raffel. ByT5: Towards a token-free future with pre-trained byte-to-byte models. *Transactions of the Association for Computational Linguistics*, 10:291–306, 2022. arXiv:2105.13626.

[20] Jonathan H. Clark, Dan Garrette, Iulia Turc, and John Wieting. CANINE: Pre-training an efficient tokenization-free encoder for language representation. *Transactions of the Association for Computational Linguistics*, 10:73–91, 2022.

[21] A. Pagnoni, R. Pasunuru, P. Rodriguez, J. Nguyen, B. Muller, M. Li, C. Zhou, L. Yu, J. Weston, L. Zettlemoyer, G. Ghosh, M. Lewis, A. Holtzman, and S. Iyer. Byte latent transformer: Patches scale better than tokens. arXiv preprint arXiv:2412.09871, 2024.

[22] Y. Shaham and O. Levy. Neural machine translation without embeddings. In *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics (NAACL)*, 2021.